

Fine-Tuning Qwen2.5-Coder-14B for Natural-Language $\rightarrow$ GeoGebra Geometry Construction

Training Report (Kaggle, QLoRA)

Aditya Dhull

June 25, 2026

Abstract

This report documents the fine-tuning of **Qwen2.5-Coder-14B-Instruct** to translate natural-language Olympiad-geometry problems into ordered **GeoGebra** construction programs encoded as JSON, which render as interactive diagrams. The model was trained with **QLoRA** on a free **Kaggle** GPU (NVIDIA T4, 16 GB) over a curated and paraphrase-augmented dataset of geometry problems. Training completed in roughly two hours. On 31 held-out problems the model produced **100% syntactically valid JSON** and **71% fully renderable, non-degenerate figures** with no repair pass — exceeding the earlier 3B baseline (65% *with* a repair loop). We describe the dataset, the model and hyperparameters, the training environment and procedure, the results and failure analysis, the deployment constraints, and the steps to reproduce.

1 Problem and Approach

The system maps a natural-language geometry problem to an *ordered* list of GeoGebra construction steps (JSON), each step being a named object with a native GeoGebra command that references previously defined names. Rendering the steps in a GeoGebra applet yields an interactive, draggable diagram. The schema for one step is:

```
{ "name": <unique id>, "type": "final" | "construction",  
  "class": point|line|segment|ray|circle|conic|locus|number|angle,  
  "label": <optional caption>, "command": <native GeoGebra syntax> }
```

final objects are shown; **construction** objects are hidden scaffolding. A *visibility rule* enforces that every entity named in the problem statement is marked **final** (and thus visible).

A prior fine-tune of **Qwen2.5-Coder-3B** (on a 6 GB GPU) learned the output *format* but frequently emitted incorrect GeoGebra commands (e.g. `Incircle(I, line)` instead of `Incircle(A,B,C)`). Because such command errors stem from the small base model’s limited pretrained knowledge rather than from a lack of data, we scaled the base model to 14B parameters; this report covers that 14B run.

2 Dataset

Composition. The corpus contains **232 distinct problems**: 82 hand-curated “gold” problems and 150 problems generated and then verified inside a real (headless) GeoGebra engine for both renderability and the geometric property they assert.

Split. A deterministic, source-stratified split (seed 42, $\sim 13\%$ eval) yields **201 training** and **31 evaluation** problems (the eval set comprises 11 gold + 20 generated and is never trained on).

Augmentation. The *training* split is paraphrase-augmented: each statement is restated in meaning-preserving ways (synonyms, openers, British/American spelling) while the construction target is unchanged. This expands the training set $201 \rightarrow 658$ and improves robustness to phrasing. The eval split is left original to keep the measurement honest.

Format. Each example is a chat triple — a fixed *system* prompt describing the schema and the visibility rule, a *user* turn (the problem statement), and an *assistant* turn (the JSON construction). The longest example is $\approx 1,295$ tokens.

Table 1: Dataset composition and split.

	Train	Eval
Gold (hand-curated)	71	11
Generated (validated)	130	20
Distinct problems	201	31
After paraphrase aug.	658	31 (unchanged)

3 Model and Method

Base model. Qwen2.5-Coder-14B-Instruct, loaded in 4-bit (unsloth/Qwen2.5-Coder-14B-Instruct-bnb) chosen for its strong code/structured-output ability and permissive Apache-2.0 license.

Method. **QLoRA:** low-rank adapters trained on top of the frozen 4-bit base, via the Unsloth library (which provides the memory optimizations that let a 14B fit a single 16 GB GPU). Only the response (assistant JSON) contributes to the loss; the prompt is masked.

Trainable parameters. **137,625,600** of **14,907,659,264** ($\approx 0.92\%$).

Epoch choice. Because the augmentation makes each construction appear ~ 3 times per epoch, 2 epochs (~ 6 exposures per construction) is the sweet spot: it halves runtime versus 4 epochs *and* reduces overfitting on the 232 distinct constructions.

4 Training Environment and Procedure

Training ran on a Kaggle notebook with a single GPU (Unsloth is single-GPU; the second T4 in a “T4 $\times 2$ ” instance cannot accelerate a 14B that already needs Unsloth’s memory savings to fit one 16 GB card).

Runtime and loss. Training took **6,917 s** (≈ 1 h 55 m) for 166 steps (≈ 32 – 42 s/step). The mean training loss was **0.0346**; per-step loss fell smoothly from ~ 0.16 to ~ 0.006 .

Table 2: Training hyperparameters.

Setting	Value
LoRA rank r / α / dropout	32 / 32 / 0
LoRA target modules	q, k, v, o, gate, up, down
Max sequence length	1536
Per-device batch size	1
Gradient accumulation	8 (effective batch 8)
Epochs	2
Total optimizer steps	166
Learning rate / schedule	2×10^{-4} / cosine
Warmup ratio	0.05
Optimizer	adamw_8bit
Weight decay	0.01
Precision	fp16 (T4 has no bf16)
Gradient checkpointing	enabled (Unsloth)
Loss masking	response-only (ChatML markers)

Table 3: Software / hardware environment.

GPU	NVIDIA Tesla T4, 14.56 GB usable, compute 7.5
Peak GPU memory	≈ 12.7 GB
PyTorch / CUDA	2.10.0+cu128 / 12.8
Triton	3.6.0
Transformers	5.5.0
Unsloth	2026.6.9
Attention backend	xFormers 0.0.35 (FA2 unavailable)

5 Results

The trained model was evaluated on the 31 held-out problems by generating a construction for each (greedy decoding, up to 1536 new tokens), then rendering every construction in a headless GeoGebra engine across 7 random parameterizations. A construction counts as “valid” only if *all* trials render with no broken steps and no degeneracy (points at infinity, collapse, mass coincidence).

The 14B at **71% raw** surpasses the 3B’s **65% with** a repair pass; the repair loop was not applied to the 14B. On simple single-construction prompts (incircle and contact triangle, the three altitudes, two tangents from an external point) the 14B produced correct, complete figures.

Failure analysis (9 of 31). The remaining failures are command-level issues:

- **Hallucinated commands** (plausible names that are not GeoGebra): `Incenter(...)`, `PerpendicularFoot(...)`, `Reflection(...)` (the correct name is `Reflect`).
- **Malformed commands**: `Intersect(Circle(O2,P1), 1)` (missing second object), `AngleBisector(A, B)` (needs three points), `Intersect(incircle, seg)` (missing index).
- **Undefined references**: e.g. `Line(A, Aprime)` with `Aprime` never defined.

Most of these are mechanical and are exactly what the generate→validate→repair loop corrects (feed the failing step and error back); applying it is expected to lift the rate toward $\sim 80\%$.

Table 4: Representative training-loss trajectory.

Epoch	Loss
0.12	0.1606
0.49	0.0578
0.85	0.0254
1.09	0.0143
1.57	0.0078
1.94	0.0070

Table 5: Functional evaluation on 31 held-out problems (14B vs. 3B baseline; identical split).

Model	Parseable JSON	Renders & valid
Qwen2.5-Coder-3B	100%	61% raw / 65% with repair loop
Qwen2.5-Coder-14B	100%	71% raw (no repair)

6 Deployment

Producing a single quantized GGUF for local serving requires merging the LoRA into a 16-bit copy of the 14B and quantizing it, which peaks at ≈ 58 GB of disk — exceeding Kaggle’s ≈ 57 GB, so the GGUF export fails with “no space left on device.” The training itself is unaffected; the LoRA adapter (≈ 290 MB) saves successfully. The model is therefore served by **loading the 4-bit base + adapter** in a Kaggle/Colab inference notebook (no GGUF needed). Generated JSON is pasted into a single-file GeoGebra renderer to view the interactive diagram.

7 Reproducibility

The pipeline is script-driven:

- `prepare_data.py` — merges the 232 problems, applies the visibility rule, augments the train split, and writes `train.jsonl` (658) / `eval.jsonl` (31).
- `augment.py` — meaning-preserving paraphrasing of statements.
- `qlora_train_14b.py` — the QLoRA training described above.
- Inference cells — load the adapter and generate JSON for new problems.

A minimal inference recipe (Kaggle: GPU + Internet on, `geom14b-adapter` dataset attached):

```
from unsloth import FastLanguageModel
import glob, json
ADAPTER = glob.glob("/kaggle/input/**/adapter_config.json", recursive=True)[0].rsplit(
    "/", 1)[0]
m, tok = FastLanguageModel.from_pretrained(ADAPTER, max_seq_length=2048, load_in_4bit=
    True)
FastLanguageModel.for_inference(m)
# build chat prompt (system schema + user problem), greedy generate, parse JSON,
    lowercase class
```

8 Limitations and Future Work

- **Data breadth.** Only 232 *distinct* constructions; augmentation adds phrasing variety, not new construction types. More distinct, validated problems would widen coverage.

- **Command hallucinations.** A short “use exactly these GeoGebra commands” hint in the prompt, and/or more data, would suppress invented names like **Reflection**.
- **Repair loop.** Not yet wired into the 14B inference path; doing so should raise the usable rate to $\sim 80\%$.
- **Logical correctness.** The validator checks *renderability*, not whether the figure faithfully matches the statement; a machine-checkable goal field would close this gap.
- **Serving.** A free, always-on API for a 14B is impractical; on-demand notebook inference (or a smaller distilled model with a GGUF) is the realistic path.

9 Conclusion

Scaling the base model to Qwen2.5-Coder-14B and fine-tuning with QLoRA on a free Kaggle T4 produced a model that reliably emits valid, complete GeoGebra constructions for most held-out problems (100% parseable, 71% renderable raw), clearly improving on the 3B baseline and fixing the dominant command-error mode at little additional cost. The remaining gap is addressable with the existing repair loop, a small prompt refinement, and additional distinct training data.